

SRV6-CORE LAB · IN DEPTH

SRv6 In Depth

A high-level overview for anyone who needs to understand what SRv6 is and why operators adopt it — followed by the details an engineer needs: the bits on the wire, the behaviours, the protocol extensions, traffic engineering, operations and security, each shown on this lab's routers.

PART 1

SRv6 at a glance

What it is, what it gives an operator, the mental model, how it compares, and its trade-offs. No prior knowledge assumed; about ten minutes.

PART 2

The details

Addressing, the SRH decoded byte by byte, network programming, compression, IS-IS and BGP extensions, TE, resilience, the Linux data plane, OAM, security, design.

Lab version 1.6.0 · 29 September 2026 · outputs captured from the running lab

Contents

PART 1 — SRV6 AT A GLANCE

1. SRv6 in one page	3
2. What it gives an operator	3
3. The mental model: network programming	4
4. Where SRv6 fits	4
5. The life of a VPN packet, in five steps	5

PART 2 — THE DETAILS

6. Addressing: the anatomy of a SID	6
7. The Segment Routing Header, bit by bit	6
8. Network programming: the behaviours	9
9. Compression: micro-SIDs (C-SIDs)	10
10. The IGP: IS-IS extensions	10
11. BGP services: carrying the service SIDs	12
12. Traffic engineering	14
13. Resilience	15
14. The Linux data plane	15
15. OAM: seeing what SRv6 is doing	16
16. Security	16
17. Design and migration	17
18. Troubleshooting checklist	18
Appendix A — command cheat sheet	18
Appendix B — the standards	19

Where this fits. This is the second of the lab's guides. *SRv6 Lab Guide* (docs/srv6-lab-guide.pdf) is the gentle introduction and tour; this one assumes you are ready for more. Every output quoted here was captured from the running lab when the document was built; function values in SIDs (e000 , e001 ...) are allocated by the routers and can differ in your lab.

SRv6 at a glance

1. SRv6 in one page

SRv6 (Segment Routing over IPv6) is a way to run a service-provider network — customer VPNs, chosen paths, fast recovery — using nothing but IPv6 forwarding.

- **The source decides the path.** The router where a packet enters writes an ordered list of instructions into it. Routers along the way execute the next instruction and keep no state about the flow.
- **An instruction is an IPv6 address.** Each instruction, a *SID*, is a 128-bit address: the first part routes the packet to a router (the *locator*), the rest tells that router what to do (the *function*).
- **The core needs only IPv6.** A router that is not the target of the current instruction just forwards the packet like any IPv6 packet. No labels, no LDP, no RSVP.
- **Protocols you already run carry it.** IS-IS (or OSPFv3) announces the locators; BGP announces which SID leads to which customer prefix.
- **Micro-SIDs keep it compact.** Several instructions fit in one address, so a path through several routers costs no more header than a single hop.

In this lab: four PEs and three P routers run SRv6 with micro-SIDs; two tenants get isolated IPv4 and IPv6 VPNs across it; the middle P router (p2) carries every tenant's traffic without a single VRF, label or customer route.

2. What it gives an operator

Need	How SRv6 meets it	Where to see it in the lab
Separate customers on one core (L3VPN, L2VPN)	a <i>service SID</i> per customer VRF on each PE; BGP advertises it with the customer's routes	two tenants, one End.DT46 SID each per PE
Choose the path for some traffic (traffic engineering)	put waypoints into the segment list at the ingress; no state on the waypoints	<code>lab.sh steer</code> , the portal's steering map
Recover fast when a link fails	BFD for detection; TI-LFA backup paths expressed as segment lists	BFD on every core link: 4 packets lost on a cut
Simplify the network	one forwarding plane (IPv6), one IGP, BGP — no LDP/RSVP to run and debug	the core runs IS-IS and nothing else
Scale	stateless core; summarisable locators; per-VRF SIDs	p2 holds 7 locators, not thousands of customer routes
Program services	new behaviours are new functions on a SID, not new protocols	End.DT46 serves IPv4 and IPv6 with one SID

3. The mental model: network programming

The idea behind SRv6 is called **network programming** (RFC 8986): treat the network like a computer, and the packet like a program.

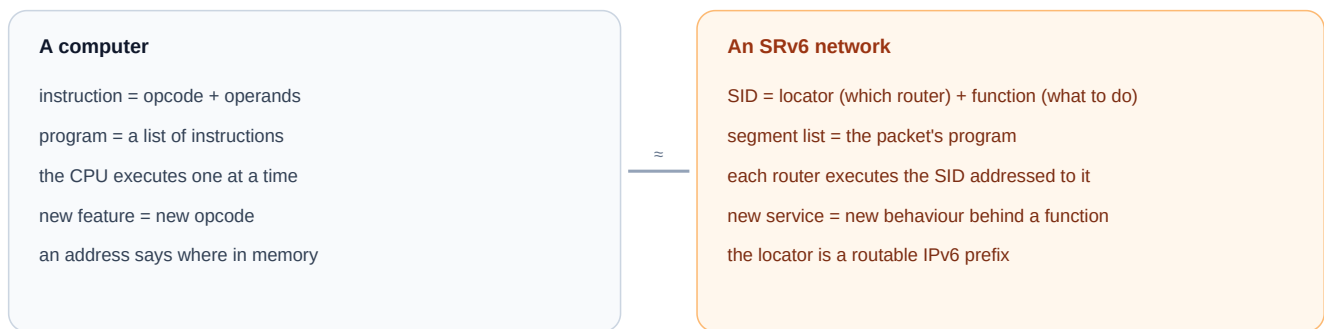


Figure 1 — the analogy that gives SRv6 its flexibility: services are "instructions" defined by routers, and packets carry "programs" built from them.

A router **defines** instructions by binding behaviours to SIDs in its own locator ("function e001 on pe3 means: decapsulate into VRF tenant-a"). It **publishes** them (IS-IS for topology instructions, BGP for service instructions). An ingress router **composes** a program from published instructions and puts it into the packet.

4. Where SRv6 fits

	Classic MPLS (LDP / RSVP-TE)	SR-MPLS	SRv6
Data plane	MPLS labels	MPLS labels	IPv6
Core must support	MPLS on every hop	MPLS on every hop	IPv6 on every hop; SRv6 only where a SID is executed
Transport signalling	LDP, RSVP-TE	none (IGP extensions)	none (IGP extensions)
Path state in the core	per tunnel (RSVP-TE)	none	none
Header per instruction	4 bytes	4 bytes	16 bytes, or 2 with micro-SIDs
Crosses a plain IP network	no (needs tunnelling)	no	yes
Maturity	decades	mature	standardised, broad vendor support, still maturing in tooling and hardware

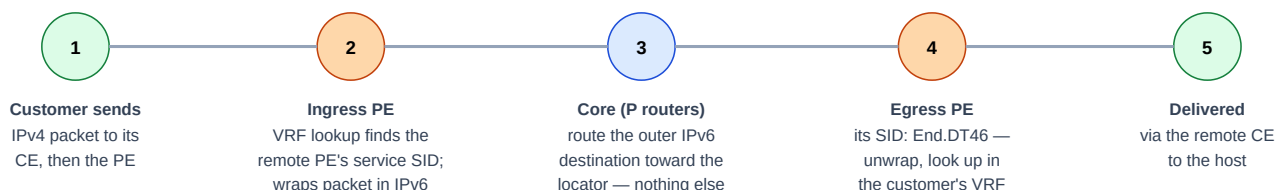
When SRv6 is a good fit: new or refreshed networks that are IPv6-capable end to end; operators who want one forwarding plane from data centre to WAN; designs where the core should stay ignorant of services; networks where hosts or data-centre fabrics might one day program paths themselves.

The trade-offs to weigh:

- **Header overhead.** An outer IPv6 header is 40 bytes, plus 8 bytes of SRH and 16 per full SID. Core MTU must allow for it (this lab uses 9000). Micro-SIDs and reduced encapsulation keep it small.

- **Hardware support.** Forwarding at line rate depends on how many SIDs a chip can push or inspect. Routers advertise these limits (Maximum SID Depths) so the controller or ingress stays within them.
- **Security boundary.** Because SIDs are plain IPv6 addresses, the SRv6 domain must be protected at its edges (section 16).
- **Operational familiarity.** Tools and habits built around MPLS (label tables, LSP ping) have SRv6 equivalents, but teams need to learn them.

5. The life of a VPN packet, in five steps



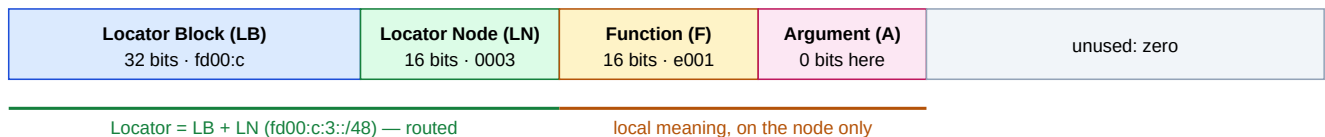
In the lab: dc1-h1 → ce1 → pe1 → p2 → pe3 → ce3 → dc3-h1. The deep version of every step is in Part 2.

Figure 2 — the whole service in one line. Part 2 opens each box.

The details

6. Addressing: the anatomy of a SID

A SID is 128 bits divided into up to four fields. RFC 8986 names them, and the routing protocols carry their lengths so every router interprets SIDs the same way:



The routing protocols carry these lengths as a "SID Structure": in IS-IS and BGP captures below you will see Locator Block length 32, Locator Node length 16, Function length 16, Argument length 0
An Argument carries per-packet data for some behaviours (for example which ports of a broadcast domain to flood to); this lab's behaviours need none.

Figure 3 — the SID fd00:c:3:e001:: (pe3, tenant-a's End.DT46) with its four fields.

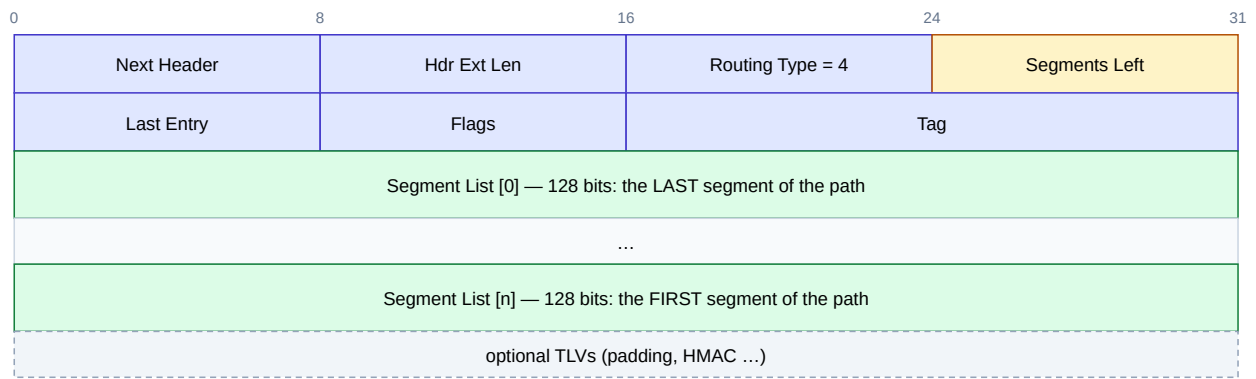
Planning locators

Decision	The lab's choice	Why it matters
Address space	fd00:c::/32 (ULA)	SIDs should come from a block that never leaves the domain; ULA or a dedicated slice of the operator's space
Block length	32 bits	Shared by all nodes; with micro-SIDs, the shorter the block the more micro-SIDs fit in one address
Node length	16 bits	65 536 possible node IDs; also the size of one micro-SID
Locator per node	/48	Everything below it (functions, arguments) belongs to that node alone
Node IDs	PEs 1–4, P routers 11–13	Readable: fd00:c:11:: is p1
Summarisation	the whole block via each P as a fallback	a whole domain can be one prefix at a border

The same routers also have **loopbacks** (fd00:a::/48) and **link addresses** (fd00:b::/48) outside the SID block. Keeping SIDs in their own block is what makes the edge filtering in section 16 a single rule.

7. The Segment Routing Header, bit by bit

The SRH (RFC 8754) is IPv6 Routing Header type 4. Its layout, 32 bits per row:



The list is stored in reverse: the active segment is Segment List[Segments Left], and it is also copied into the IPv6 destination address.

Figure 4 — the Segment Routing Header (RFC 8754).

Field	Meaning
Next Header	what follows the SRH: 4 = IPv4, 41 = IPv6, 143 = Ethernet ...
Hdr Ext Len	SRH length in 8-byte units, not counting the first 8 bytes: $(len + 1) \times 8$ bytes
Segments Left (SL)	index of the active segment; decremented by each End-type behaviour
Last Entry	index of the last element of the segment list (n)
Flags	the O-flag (OAM, RFC 9259) marks packets whose SIDs should be punted for inspection
Tag	marks a packet as part of a class or group

A real packet, decoded

On p2, a single ping from dc1-h1 to dc3-h1 (sent with `-s 16` to keep it short) captured with `tcpdump -xx`:

```
p2$ sudo tcpdump -ni eth5 -c 1 -xx -v 'ip6 and dst net fd00:c:3::/48 and ip6 proto 43'
18:47:53.618019 IP6 (hlim 62, next-header Routing (43) payload length: 68) fd00:a::1 > fd00:c:3:e001::: RT6 (len=
2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c:3:e001::) IP (tos 0x0, ttl 63, id 1457, offset 0,
flags [DF], proto ICMP (1), length 44)
  172.20.1.2 > 172.20.3.2: ICMP echo request, id 1184, seq 0, length 24
    0x0000: 5254 00c6 0301 5254 00c6 0605 86dd 6000
    0x0010: 0000 0044 2b3e fd00 000a 0000 0000 0000
    0x0020: 0000 0000 0001 fd00 000c 0003 e001 0000
    0x0030: 0000 0000 0000 0402 0400 0000 0000 fd00
    0x0040: 000c 0003 e001 0000 0000 0000 0000 4500
    0x0050: 002c 05b1 4000 3f01 d9f3 ac14 0102 ac14
    0x0060: 0302 0800 12bb 04a0 0000 099c d708 0000
    0x0070: 0000 0000 0000 0000 0000
```

Offset	Bytes	Field	Value
0x00	5254 00c6 ... 86dd	Ethernet	next hop's MAC, p2's MAC, type 0x86dd = IPv6
0x0e	6000 0000	IPv6 version / class / flow	version 6, traffic class 0, flow label 0
0x12	0044	Payload length	68 bytes = 24 (SRH) + 44 (inner IPv4 packet)
0x14	2b	Next header	43 = Routing header: an SRH follows
0x15	3e	Hop limit	62 — the outer header counts core hops, independently of the customer's TTL
0x16	fd00 000a ... 0001	Source	fd00:a::1 — pe1's loopback, the encapsulation source
0x26	fd00 000c 0003 e001 ...	Destination	fd00:c:3:e001:: — the active SID: pe3, function e001
0x36	04	SRH next header	4 = an IPv4 packet is inside
0x37	02	Hdr Ext Len	$2 \rightarrow (2 + 1) \times 8 = 24$ bytes of SRH
0x38	04	Routing type	4 = Segment Routing
0x39	00	Segments Left	0 — this is already the last segment
0x3a	00 00 0000	Last Entry, Flags, Tag	a one-entry list, no flags, no tag
0x3e	fd00 000c 0003 e001 ...	Segment List[0]	the same SID as the destination
0x4e	4500 002c ... 3f01 ...	Inner IPv4 header	44 bytes long, TTL 0x3f = 63, protocol 1 (ICMP)
0x5a	ac14 0102 → ac14 0302	Inner addresses	172.20.1.2 → 172.20.3.2
0x62	08 00	ICMP	type 8 = echo request

Two details worth noticing. The inner TTL is 63: the host sent 64, ce1 decremented it, and pe1 encapsulated it without touching it again — the core's hops do not count against the customer's TTL. And the SRH is present even though it holds a single segment that duplicates the destination address: that is the plain `encap` mode.

Overhead and reduced encapsulation

Encapsulation	Bytes added to the customer packet
This lab: outer IPv6 + SRH with one segment	$40 + 8 + 16 = 64$
Reduced (H.Encaps.Red): the first SID only in the destination, no SRH when it is the only one	40
Three full SIDs, not compressed	$40 + 8 + 48 = 96$
The same three hops as one micro-SID carrier	64 (or 40 reduced)

The lab's core MTU of 9000 absorbs any of these; a network with a 1500-byte core would need to account for them. Newer Linux kernels offer the reduced mode as `encap.red`.

8. Network programming: the behaviours

RFC 8986 defines the standard behaviours. A **headend** behaviour puts packets into SRv6; an **endpoint** behaviour is what a node does when a packet's destination is one of its SIDs.

Behaviour	Kind	What it does
H.Encaps	headend	encapsulate in a new IPv6 header with an SRH holding the segment list
H.Encaps.Red	headend	the same, omitting the first SID from the SRH (the SRH is dropped if one SID remains)
H.Encaps.L2 / .L2.Red	headend	the same for an Ethernet frame (L2VPN)
End	endpoint	the basic waypoint: move to the next segment and forward
End.X	endpoint	as End, then forward out of a specific adjacency (a "cross-connect" to a neighbour)
End.T	endpoint	as End, then look up in a specific IPv6 table
End.DX4 / End.DX6	endpoint	decapsulate and send to a given IPv4 / IPv6 next hop (per-CE VPN)
End.DT4 / End.DT6 / End.DT46	endpoint	decapsulate and look up in a given VRF table (per-VRF VPN); DT46 handles both families
End.DX2, End.DT2U/M	endpoint	Ethernet: cross-connect, or unicast/flood in a bridge (EVPN)
End.B6.Encaps	endpoint	a Binding SID: steer the packet into another SR policy by adding a new outer header

What "End" actually does

When a packet arrives whose destination address is one of the node's End SIDs:

1. If **Segments Left > 0**: decrement it, copy the new active segment (Segment List[SL]) into the destination address, decrement the hop limit, look the new destination up in the routing table, and forward.
2. If **Segments Left = 0**: the SRH is finished; the node goes on to process whatever header comes next — which for End means the packet was addressed to the node itself.

End.DT46 is simpler still: it expects SL = 0, removes the outer IPv6 header and its SRH, and looks the inner packet's destination up in the VRF table bound to that SID.

Flavours

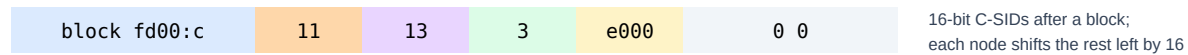
Behaviours can have **flavours** that change what happens to the SRH at the end of the path:

- **PSP** (Penultimate Segment Pop): the next-to-last node removes the SRH, so the last node receives a plain IPv6 packet.
- **USP** (Ultimate Segment Pop): the last node removes the SRH before processing the next header.
- **USD** (Ultimate Segment Decapsulation): the last node decapsulates the outer header entirely.
- **NEXT-C-SID / REPLACE-C-SID**: the compression flavours of the next section. The lab's End and End.X SIDs carry `flavors next-csid`.

9. Compression: micro-SIDs (C-SIDs)

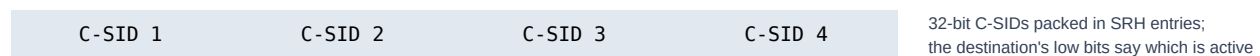
RFC 9800 standardises two ways of carrying several instructions in one 128-bit container, called **compressed SIDs**:

NEXT-C-SID ("uSID") — shift the container



When the node's C-SID is followed by only zeros, the container is used up: the node behaves like a classic End and moves to the next entry in the SRH (if any).

REPLACE-C-SID — index into the container



Both keep the destination address a routable SID at every hop. This lab uses NEXT-C-SID: the kernel shows "flavors next-csid lben 32 nflen 16" — block 32 bits, node + function 16.

Figure 5 — the two compression flavours of RFC 9800. "uSID" is the widely used name for NEXT-C-SID.

Capacity. After a 32-bit block, a 128-bit container holds six 16-bit C-SIDs. The steered path $pe1 \rightarrow p1 \rightarrow p3 \rightarrow pe3$ uses four (p1, p3, pe3, pe3's function) — `fd00:c:11:13:3:e000::`. A longer path simply continues in a second container in the SRH.

Why the destination changes. In the steering captures p1 forwards `fd00:c:13:3:e000::` and p3 forwards `fd00:c:3:e000::`: each is a normal routed address inside the next node's locator, so every router between waypoints forwards it without knowing SRv6 is involved.

10. The IGP: IS-IS extensions

RFC 9352 extends IS-IS with everything needed to make SIDs reachable and known. pe1's own link-state packet shows them all:

```

pe1$ vtysh -c 'show isis database detail pe1.00-00'
Area VyOS:
IS-IS Level-2 link-state database:
LSP ID                PduLen  SeqNumber  Chksum  Holdtime  ATT/P/OL
pe1.00-00             *    330    0x00000027  0xbdbb    358    0/0/0
  Protocols Supported: IPv4, IPv6
  Area Address: 49.0001
  Hostname: pe1
  TE Router ID: 10.3.0.11
  Router Capability: 10.3.0.11 , D:0, S:0
    Segment Routing: I:1 V:1, Global Block Base: 16000 Range: 8000
    SR Local Block Base: 15000 Range: 1000
    SR Algorithm:
      0: SPF
    SRv6: 0:0
  Extended Reachability: 0000.0000.0011.00 (Metric: 10)
    Adjacency-SID: 15001, Weight: 0, Flags: F:1 B:0, V:1, L:1, S:0, P:0
    SRv6 End.X SID: fd00:c1:e003::, Algorithm: SPF, Weight: 0, Endpoint Behavior: uA, Flags: B:0, S:0, P:0
      SRv6 SID Structure Locator Block length: 32, Locator Node length: 16, Function length: 16, Argument length: 0,
  Extended Reachability: 0000.0000.0012.00 (Metric: 10)
    Adjacency-SID: 15000, Weight: 0, Flags: F:1 B:0, V:1, L:1, S:0, P:0
    SRv6 End.X SID: fd00:c1:e002::, Algorithm: SPF, Weight: 0, Endpoint Behavior: uA, Flags: B:0, S:0, P:0
      SRv6 SID Structure Locator Block length: 32, Locator Node length: 16, Function length: 16, Argument length: 0,
  IPv4 Interface Address: 10.3.0.11
  IPv6 Reachability: fd00:c1::/48 (Metric: 0)
  IPv6 Reachability: fd00:c1::1/128 (Metric: 10)
  IPv6 Reachability: fd00:b:0:101::/64 (Metric: 10)
  IPv6 Reachability: fd00:b:0:201::/64 (Metric: 10)
  IPv6 Reachability: fd00:a::1/128 (Metric: 10)
  SRv6 Locator: fd00:c1::/48 (Metric: 0) standard
  Sub-TLVs:
    SRv6 End SID Endpoint Behavior: uN, SID value: fd00:c1::
  Sub-Sub-TLVs:
    SRv6 SID Structure Locator Block length: 32, Locator Node length: 16, Function length: 16, Argument length: 0,

```

What you see	IS-IS element	Purpose
SRv6: 0:0 under Router Capability	SRv6 Capabilities sub-TLV	"I do SRv6"; 0 = whether the node supports the SRH's OAM flag
SRv6 Locator: fd00:c1::/48 ... standard	SRv6 Locator TLV (27)	announces the locator, for algorithm 0 ("standard" = SPF); every router installs a route to it
SRv6 End SID ... uN, SID value: fd00:c1::	End SID sub-TLV	the node's own End SID (here the micro-SID form uN)
SRv6 End.X SID: fd00:c1:e003:: ... uA	End.X SID sub-TLV, under each neighbour	one per adjacency: "send out of this link" — the building block of TI-LFA and strict paths
SID Structure ... 32 / 16 / 16 / 0	SID Structure sub-sub-TLV	the LB / LN / F / A lengths of section 6
IPv6 Reachability: fd00:c1::/48 (Metric: 0)	ordinary prefix TLV	the locator again as a plain prefix, so routers that do not understand SRv6 still route to it

The same LSP also carries **SR-MPLS** information (a label block and Adjacency-SIDs as labels): FRR advertises it when segment routing is enabled, but nothing in this lab forwards on labels.

Maximum SID Depths

Every node advertises what its data plane can handle, so an ingress or a controller never builds a segment list a router cannot process:

```
pe1> show isis segment-routing srv6 node
Area VyOS:
IS-IS L1 SRv6-Nodes:

IS-IS L2 SRv6-Nodes:

System ID      Algorithm  SRH Max SL  SRH Max End Pop  SRH Max H.encaps  SRH Max End D
-----
0000.0000.0001  SPF       3           3                2                5
0000.0000.0002  SPF       3           3                2                5
0000.0000.0003  SPF       3           3                2                5
0000.0000.0004  SPF       3           3                2                5
0000.0000.0011  SPF       3           3                2                5
0000.0000.0012  SPF       3           3                2                5
0000.0000.0013  SPF       3           3                2                5
```

MSD	Meaning	Here
Max Segments Left	the largest SL value the node can process in a received SRH	3
Max End Pop	how many SIDs deep it can look when removing the SRH (PSP/USP)	3
Max H.Encaps	how many SIDs it can push when encapsulating	2
Max End D	how many SIDs deep it can look when decapsulating	5

With micro-SIDs a two-SID push covers paths of up to twelve instructions.

Algorithms and Flex-Algo

The `Algorithm: SPF` column is algorithm 0, the plain shortest path. **Flexible Algorithm** (RFC 9350) lets an operator define more — "lowest delay", "avoid links coloured red" — as algorithms 128–255. In SRv6 each algorithm gets **its own locator** on every node, so choosing a path type is simply choosing which locator's SID to put in the packet. The lab runs only algorithm 0; Flex-Algo is a natural next step (a "low-delay" locator per node).

11. BGP services: carrying the service SIDs

RFC 9252 extends BGP so a VPN route can carry the SID that leads to it. For each VRF, pe3 allocates one End.DT46 SID and attaches it to every route it exports from that VRF. The receiving PE sees:

```

pe1> show bgp ipv4 vpn 172.20.3.0/24
BGP routing table entry for 65000:103:172.20.3.0/24, version 2
not allocated
Paths: (2 available, best #1)
  Not advertised to any peer
  65003
    0.0.0.0 (metric 20) from fd00:a::11 (10.255.0.3)
      Origin IGP, metric 0, localpref 100, valid, internal, multipath, best (Neighbor IP)
      Extended Community: RT:65000:100
      Originator: 10.255.0.3, Cluster list: 10.255.0.11
      Remote labels: 917520
      Remote SID: fd00:c:3::, sid structure=[32 16 16 0 16 48]
      Last update: Tue Sep 29 11:37:16 2026
  65003
    0.0.0.0 (metric 20) from fd00:a::13 (10.255.0.3)
      Origin IGP, metric 0, localpref 100, valid, internal, multipath
      Extended Community: RT:65000:100
      Originator: 10.255.0.3, Cluster list: 10.255.0.13
      Remote labels: 917520
      Remote SID: fd00:c:3::, sid structure=[32 16 16 0 16 48]
      Last update: Tue Sep 29 11:37:16 2026

```

Where the SID travels, and "transposition"

The SID rides in the **BGP Prefix-SID attribute** (path attribute 40), in an *SRv6 L3 Service TLV* containing an *SRv6 SID Information sub-TLV* and a *SID Structure sub-sub-TLV*. To save space when thousands of routes share one locator, BGP may **transpose** part of the SID into the MPLS label field of the VPN route — the field is there anyway, and unused in SRv6. The JSON view shows exactly what was sent:

```

pe1$ vtysh -c 'show bgp ipv4 vpn 172.20.3.0/24 json'      (one path, the SID fields)
{
  "remoteSid": "fd00:c:3::",
  "remoteSidStructure": {
    "locatorBlockLen": 32,
    "locatorNodeLen": 16,
    "functionLen": 16,
    "argumentLen": 0,
    "transpositionLen": 16,
    "transpositionOffset": 48
  },
  "remoteLabel": 917520,
  "nexthops": [
    {
      "ip": "0.0.0.0",
      "hostname": "p1",
      "afi": "ipv4",
      "metric": 20,
      "accessible": true,
      "used": true
    }
  ]
}

```

Rebuilding the SID from the route

remoteSid fd00:c:3::

remoteLabel 917520

transpositionOffset 48: the bits start
at bit 48 of the SID (after LB + LN)
transpositionLength 16: 16 of them

917520 = 0xE0010 (20-bit label field)
the top 16 bits = 0xE001 → the function

fd00:c:3

e001

::

= pe3's tenant-a End.DT46 SID

Figure 6 — transposition decoded: the locator in the SID field, the function in the label, recombined by the receiving PE.

Design choices in the service layer

Choice	Options	The lab
SID allocation	per VRF (End.DT4/6/46: one SID, lookup in the VRF) or per CE / per next hop (End.DX4/6: no lookup, straight to a CE)	per VRF, one End.DT46 for both families
IPv4 VPN over an IPv6 core	the VPN routes carry an IPv6 next hop (RFC 8950, <code>extended-nexthop</code>)	yes: PEs peer over IPv6 loopbacks
Route distribution	full mesh or route reflectors	two reflectors (p1, p3) with different cluster IDs, so every route arrives twice and losing one changes nothing
Route separation	RD per PE per VRF, RT per VRF	RD 65000:10n / 20n, RT 65000:100 / 200

12. Traffic engineering

The SR Policy model

RFC 9256 defines an **SR Policy** as the general way to steer traffic onto a chosen path:

SR Policy = (headend, color, endpoint)

color: an intent, e.g. 100 = "low delay"

endpoint: the egress PE

candidate paths, each with a preference:

- explicit segment lists (configured / controller)
- dynamic (computed for a metric / constraints)

the best valid candidate path is active

Binding SID: one SID that stands for the policy

How traffic gets into a policy

Automated steering: a BGP route carrying the Color extended community C, with next hop E, uses policy (C, E) if it exists.

On-demand next hop (ODN): the policy is created when the first route with that color arrives — intent travels with the route.

Per-flow or per-destination steering, or via the Binding SID from another domain.

Figure 7 — the SR Policy model: intent (color) and destination (endpoint) select a path; the path is a segment list.

What the lab does, and how it relates

The lab's steering is deliberately minimal: `tools/steer.py` installs a **static route in the tenant's VRF** whose next hop is an SRv6 encapsulation. It is the data-plane result an SR Policy would produce, without the policy machinery:

```
$ ./lab.sh steer add pe1 tenant-b 172.21.3.0/24 p1 p3
pe1: tenant-b 172.21.3.0/24 -> p1 -> p3 -> pe3 segments fd00:c:11:13:3:e000:: (out eth1) [uSID: one compressed segment]
172.21.3.0/24 nhid 142 encap seg6 mode encap segs 1 [ fd00:c:11:13:3:e000:: ] dev eth1 proto static metric 20
```

Building the carrier: take the block `fd00:c`, append the uN C-SID of each waypoint (p1 = `11`, p3 = `13`), then the egress PE's node C-SID and function taken from its tenant-b End.DT46 SID (pe3 = `3`, `e000`, read live with `steer.py sid`). The portal's steering map computes the same waypoint path from the model and draws it next to the IGP's, and its **Measure** button compares the delay of both.

13. Resilience

Mechanism	What it does	In this lab
BFD (RFC 5880)	detects a dead neighbour in milliseconds, independent of the link's carrier	300 ms × 3 on every core adjacency
IGP reconvergence	recomputes shortest paths and reinstalls routes	IS-IS with BFD: ~4 packets lost at 0.2 s intervals on a silent cut
TI-LFA	precomputes, for every destination, a loop-free backup path as a segment list (often an End.X SID to force a link), installed before any failure	not configured — a natural extension
Microloop avoidance	temporarily steers traffic along the post-convergence path while routers update at different times	not configured
Redundant route reflectors	BGP keeps a second copy of every VPN route	p1 and p3; suite 06 shuts one and nothing changes

TI-LFA is where SRv6's End.X SIDs earn their keep: a backup path such as "go to p3, then out of p3's link to pe3" is just two SIDs, with no pre-signalled tunnel, and covers any topology.

14. The Linux data plane

Every router in the lab is VyOS; the forwarding happens in the Linux kernel through two *lightweight tunnel* types:

- `seg6` — headend: a route whose next hop is an encapsulation (`encap seg6 mode encap segs ...`), used in the VRF tables.
- `seg6local` — endpoint: a route for a local SID with an action (`End , End.X , End.DT46 ...`), installed by FRR for IS-IS and BGP SIDs.

A few kernel details show up in practice:

```
pe1$ cat /proc/sys/net/ipv6/conf/*/seg6_enabled ; sudo ip sr tunsrc show
net.ipv6.conf.all.seg6_enabled = 1
net.ipv6.conf.eth1.seg6_enabled = 1
net.ipv6.conf.eth2.seg6_enabled = 1
net.ipv6.conf.eth3.seg6_enabled = 0
tunsrc addr fd00:a::1
```

- `seg6_enabled` decides, per interface, whether packets carrying an SRH are accepted. The core links (eth1, eth2) accept them; the customer-facing eth3 does not — a first line of defence (section 16).
- `tunsrc` is the source address of every encapsulation: pe1's loopback, which is why the captures show `fd00:a::1 > ...`.
- The SIDs live on a dummy interface (`dum0`) addressed with a /128, so the locator's connected route never outranks the uN route.

The VRF lookup quirk

For **forwarded** packets (those arriving from a CE), the kernel looks up the encapsulation's outer destination in the **VRF's** table, not the global one. A PE therefore needs routes to the remote locators inside every tenant

VRF. The lab installs them as static routes that follow the IGP's shortest path, plus the whole block as a fallback:

```
pe1$ ip -6 route show vrf tenant-a | grep fd00:c:
fd00:c:2::/48 nhid 67 proto static metric 20 pref medium
fd00:c:3::/48 nhid 62 via fe80::5054:ff:fec6:603 dev eth2 proto static metric 20 pref medium
fd00:c:4::/48 nhid 62 via fe80::5054:ff:fec6:603 dev eth2 proto static metric 20 pref medium
fd00:c:11::/48 nhid 61 via fe80::5054:ff:fec6:503 dev eth1 proto static metric 20 pref medium
fd00:c:12::/48 nhid 62 via fe80::5054:ff:fec6:603 dev eth2 proto static metric 20 pref medium
fd00:c:13::/48 nhid 67 proto static metric 20 pref medium
fd00:c::/32 nhid 67 proto static metric 20 pref medium
               nexthop encap seg6 mode encap segs 1 [ fd00:c:2:e001:: ] via fe80::5054:ff:fec6:503 dev eth1 weight 1
```

15. OAM: seeing what SRv6 is doing

Ping and traceroute. A locator is routed, but a SID is an instruction, not an interface address. A plain ping to a *SID* is handed to the SID's behaviour; the kernel's End implementation does not deliver it to the local stack, so it gets no answer — while the router's loopback, over the same path, answers normally:

```
pe1$ ping -6 -c 3 fd00:c:3::                (pe3's uN SID)

--- fd00:c:3:: ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 609ms
pe1$ traceroute -6 -n fd00:c:3::
traceroute to fd00:c:3:: (fd00:c:3::), 30 hops max, 80 byte packets
 1 fd00:b:0:201::1 1.163 ms
 2 *
 3 *
 4 *
 5 *
pe1$ traceroute -6 -n fd00:a::3                (pe3's loopback)
traceroute to fd00:a::3 (fd00:a::3), 30 hops max, 80 byte packets
 1 fd00:b:0:201::1 0.382 ms
 2 fd00:a::3 0.629 ms
```

RFC 9259 standardises SRv6 OAM: the O-flag in the SRH asks each SID's owner to punt a copy for inspection, and ping/traceroute to SIDs where implementations support it.

Capturing. Useful tcpdump filters in an SRv6 core:

Filter	Catches
ip6 proto 43	IPv6 packets with a Routing header directly after the base header (SRH)
ip6 and dst net fd00:c::/32	everything addressed to a SID in the lab's block (with or without SRH)
ip6 and dst net fd00:c:3::/48	everything heading for pe3's SIDs

Measuring. Delay and loss per path can be measured with STAMP (RFC 8762) probes carried along a segment list. The lab measures from the hosts (the portal's SLA probes) and from the PE inside a VRF (the steering map's Measure), and the looking glass records every route change so a path can be read at any past moment.

16. Security

SRv6's strength — SIDs are ordinary IPv6 addresses — is also its main risk: a packet from outside that is addressed to a SID, or carries an SRH, could make routers execute instructions. RFC 8754 therefore defines

an **SR domain** with a trusted boundary:

Measure	What it prevents	In this lab
Drop, at every domain edge, packets whose destination is in the SID block	outsiders invoking SIDs (e.g. decapsulating into a VRF)	the SID block is private ULA and never advertised outside; no explicit edge ACL
Do not accept SRHs on customer-facing interfaces	outsiders injecting segment lists	<code>seg6_enabled = 0</code> on the CE-facing ports
Keep the SID block out of every external routing table	discovery and reachability from outside	the block lives only in IS-IS and the VRF leaks
HMAC TLV in the SRH (RFC 8754)	tampering with a segment list across a less trusted segment	not used
Separate VRFs per customer; RT discipline	leaks between customers	the tests prove no cross-tenant route or packet

A single infrastructure ACL on "destination in `fd00:c::/32` from outside the domain → drop" is the standard edge rule, and the reason to keep all SIDs in one block.

17. Design and migration

Design checklist.

- **MTU:** size the core for the largest customer packet plus 40–96 bytes of encapsulation.
- **Locators:** one block for the domain, one locator per node (per algorithm with Flex-Algo), node IDs sized to fit your micro-SID width.
- **Hardware:** check the Maximum SID Depths against the longest segment list you intend to use; prefer micro-SIDs.
- **Service SIDs:** per VRF unless a design needs per-CE forwarding.
- **Route reflection:** at least two reflectors; separate them from the forwarding path where possible (p2 in the lab carries traffic and no BGP).
- **Protection:** BFD everywhere; TI-LFA for sub-50 ms protection.
- **Security:** edge filters on the SID block, SRH acceptance only on core interfaces.

Migrating from MPLS. SRv6 and MPLS can run side by side: introduce IPv6 and SRv6 locators on the existing core, let new PEs speak SRv6 services while old ones keep MPLS, and interconnect the two service domains at gateway PEs that terminate one and originate the other. Services move PE by PE; LDP and RSVP are removed once nothing depends on them.

18. Troubleshooting checklist

Symptom	Check	Command in the lab
No VPN routes from a remote PE	BGP session to the reflectors; RT import	<code>show bgp ipv4 vpn summary</code> , <code>show bgp ipv4 vpn <prefix></code>
VPN route present, no SRv6 encapsulation in the VRF	the Prefix-SID was received; the SID is resolvable	<code>ip route show vrf <tenant> <prefix></code>
Encapsulation present, packets dropped at the ingress	the outer lookup in the VRF (the quirk)	<code>ip -6 route show vrf <tenant> grep fd00:c: , nstat grep OutNoRoutes</code>
Packets leave the ingress, never reach the egress	the remote locator in the core's routing tables	<code>show ipv6 route isis</code> , <code>show isis neighbor</code>
Packets reach the egress PE, not the customer	the End.DT46 SID exists and points at the right VRF	<code>ip -6 route show grep seg6local</code>
A steered path is not taken	the policy route and its carrier	<code>lab.sh steer show</code> , captures on the waypoints
Intermittent loss on a path	adjacency flaps, BFD	<code>show bfd peers brief</code> , the portal's SLA history, the looking glass's History

Appendix A — command cheat sheet

Task	VyOS / FRR	Linux (on the router)
Locators	<code>show segment-routing srv6 locator</code>	<code>ip -6 addr show dev dum0</code>
Local SIDs	<code>vtysh -c 'show ipv6 route'</code> (seg6local entries)	<code>ip -6 route show grep seg6local</code>
SRv6 nodes and MSDs	<code>show isis segment-routing srv6 node</code>	—
An LSP with its SRv6 TLVs	<code>vtysh -c 'show isis database detail <node>.00-00'</code>	—
VPN routes and SIDs	<code>show bgp ipv4 vpn <prefix></code> (add json for the SID structure)	—
A VRF's forwarding	<code>show ip route vrf <tenant></code>	<code>ip route show vrf <tenant></code>
SRH on the wire	—	<code>sudo tcpdump -ni ethN -vv 'ip6 proto 43'</code>
Encapsulation source	—	<code>sudo ip sr tunsrsrc show</code>
Steering	<code>./lab.sh steer add del show sid</code>	—

Appendix B — the standards

RFC	Title (short)	Covered in
8402	Segment Routing Architecture	§1–3
8754	IPv6 Segment Routing Header (SRH)	§7, §16
8986	SRv6 Network Programming (behaviours, flavours)	§3, §8
9800	Compressed SRv6 Segment List Encoding (C-SID, uSID)	§9
9352	IS-IS Extensions to Support SRv6	§10
9513	OSPFv3 Extensions for SRv6	§10 (the OSPF equivalent)
9350	IGP Flexible Algorithm	§10
9252	BGP Overlay Services Based on SRv6	§11
8950	IPv4 NLRI with an IPv6 Next Hop	§11
4364	BGP/MPLS IP VPNs (VRF, RD, RT)	§11
9256	Segment Routing Policy Architecture	§12
5880	Bidirectional Forwarding Detection	§13
9259	SRv6 OAM	§15
8762	STAMP (active measurement)	§15